# One rule, two languages

C++/Kokkos and Rust ports of WAVEWATCH III that must give the Fortran's bits

Matheus Hoffmann ORCID 0009-0009-1056-7661
*Escola Politécnica, Universidade Federal do Rio de Janeiro, Brazil*

Pedro Veras Guimarães ORCID 0000-0002-1686-4722
*Co-advisor of the first author. LabECO, Departamento de Engenharia Mecânica, Universidade Federal de Santa Catarina, Brazil*

Pedro Cruz ORCID 0000-0002-2019-133X
*Advisor of the first author. Grupo de Teleinformática e Automação (GTA), DEL, Escola Politécnica, Universidade Federal do Rio de Janeiro, Brazil*

11 November 2026 · v0 (preprint, not peer reviewed) · rev. 76dd794+270

**Abstract.** We report two ports of the WAVEWATCH III (WW3) spectral wave model. Coding agents wrote both ports, and both ports have one acceptance rule: the port must give the same bits as the Fortran. A tolerance is a diagnostic, never the verdict. *ww3-gpu* keeps WW3, and the Fortran calls C++ written with Kokkos, behind one run-time flag for each routine. Its reference is the Fortran path of the same executable. *chaceon* replaces WW3 with Rust, compiled for the CPU, for SPIR-V (rust-gpu) and for PTX (Rust-CUDA). No Fortran runs in its process, and its reference is a WW3 that is compiled separately. ww3-gpu ports the discrete interaction approximation (DIA) of the four-wave transfer. On the CPU, it is byte-identical on all 100 serial and 25 OpenMP entries of the WW3 regression matrix that select it. The DIA takes 48.4 % of single-thread self time on a case of 9 520 points. chaceon runs `ww3_grid`, `ww3_strt`, `ww3_prep` and `ww3_shel` without Fortran, and 96 output files are byte-identical to the reference. Neither port has run the whole model on a real GPU, and this note claims no speed-up. We compare how each project defines and checks its evidence.

**Keywords:** WAVEWATCH III; bit-for-bit reproducibility; Kokkos; Rust; GPU porting; coding agents.

## 0 Context: an ocean model ported by a language model

Koldunov et al. (Koldunov et al. 2026) report the port of a whole ocean model by a large language model. The model is FESOM2, an ocean and sea-ice model, and the authors are at the Alfred Wegener Institute. Claude Code, with Claude Opus 4.7, translated it from Fortran to C and then from C to C++/Kokkos. The people set the strategy, reviewed the plans and the diffs, and found physics and validation errors. The agent read, translated, built the test harnesses, debugged, profiled and wrote the documentation.

The C stage is the centre of their method. The clean C code is the reference, and each Kokkos kernel keeps its C original as a "twin". The acceptance rule changes at each step. The C model is compared with the Fortran by statistics: over five years, the RMS difference of the mean is 0.006 °C in sea-surface temperature. Only the Kokkos Serial model is byte-identical, and only to the C model. The authors write that bit-for-bit agreement between Fortran and C "across compilers and arithmetic orderings is neither attainable nor necessary". On the GPU, they accept differences up to a recorded ceiling for each field.

The successor of WW3 is also in progress. NCEP Office Note 525 (Tolman 2025) chose a bottom-up rewrite, with a C++ core first and a Rust core later ⚠. The same report demotes Fortran to a solver-only language ⚠. The WAVEWATCH IV repository (NOAA Environmental Modeling Center 2026) is pre-alpha: on 2026-11-11 its `develop` branch had 40 commits, from 2025-11-17 to 2026-10-16 (`875f49a`). Its 97 source files are C++ (66 `.cpp`, 31 `.h`), and no physics kernel of WW3 is in it yet.

Thus the core kernels of the wave model stay in Fortran for some years ⚠. This is the first reason why the first author will continue to maintain ww3-gpu and chaceon. They make the Fortran kernels of WW3 run on GPUs now, without a change to their bits.

This note starts from that result and asks a narrower question. Can an agent port a spectral wave model, and give the bits of the Fortran itself? The two ports below have no C stage, and their reference is the Fortran. They also use two practices that FESOM2 reports: a strictly literal translation, and a `file:line` with the literal value for each statement about the Fortran.

## 1 Why bit for bit

WW3 integrates the action balance equation

$$\frac{\partial N}{\partial t} + \nabla \cdot (\dot{\mathbf{c}}\, N) = \frac{S}{\sigma},$$

where the source term $S$ is the sum of wind input, dissipation, nonlinear transfer and bottom terms. A port that changes the order of a floating-point sum changes $N$. A forecast system that was validated against observations must then be validated again. Thus both projects keep the arithmetic of the Fortran. They do not reorder a reduction, they do not use a fused multiply-add (`-ffp-contract=off` on both sides), and they do not widen a tolerance. A difference is a defect of the port until two runs of the Fortran itself give different bytes in the same file.

## 2 Two shapes for one rule

Table 1: The two ports. Sources: chaceon `docs/WW3_GPU_PARITY_202611.md` (2026-11-09) and `README.md` at `0c098fe` (Hoffmann 2026a); ww3-gpu `kokkos/PORT_STATUS.md` at `58167d7` (Hoffmann 2026c).

| | ww3-gpu (C++/Kokkos) | chaceon (Rust) |
|---|---|---|
| Shape | Inside WW3: the Fortran calls C++ through a `bind(C)` shim, one flag for each routine | Replaces WW3: no Fortran in a chaceon process |
| Reference | NOAA-EMC/WW3 `03adda91`, through a fork that adds 5 687 lines and removes none | MetOcean WW3 7.14, `92e2cf66` |
| CPU parity against | the Fortran path of the same executable | a reference that is compiled separately |
| Device targets | Kokkos 5.2.2, CUDA backend (`sm_89`); OpenMP on the CPU | rust-gpu (SPIR-V, Vulkan) and Rust-CUDA (PTX) |
| Spectrum resident on the device | yes, on the OpenMP backend | yes, in SPIR-V on CPU Vulkan (lavapipe) |
| Size of the port | 62 257 lines of C++, 355 files (2026-10-14) | 29 229 lines of Rust port, 5 936 lines of tools |
| Formal methods | SMT queries, exhaustive sweeps | one Lean 4 proof about `f32` division |
| Licence | MIT, LGPL-3.0 kernels | MPL-2.0 |

Table 2: The two repositories. Source: the GitHub REST API (`repos/h0ffmann/<repo>` for `created_at` and `pushed_at`, `commits?sha=main` for the commits), read on 2026-11-11 at 03:35 UTC. The first commit of ww3-gpu is older than the repository because it was made before the first push.

| Metric | ww3-gpu | chaceon |
|---|---|---|
| Repository created | 2026-09-12 01:31 UTC | 2026-10-23 03:58 UTC |
| First commit on `main` | 2026-09-11 13:47 UTC, `b43a6d3` | 2026-10-23 03:58 UTC, `3bad31a` |
| Last commit on `main` | 2026-11-11 03:16 UTC, `efc0211` | 2026-11-09 08:53 UTC, `0c098fe` |
| Last push, any branch | 2026-11-11 03:31 UTC | 2026-11-09 08:53 UTC |
| Commits on `main` | 800 | 110 |

The Kokkos route keeps every WW3 program and replaces one routine at a time. Thus one regtest can isolate the effect of one routine. The Rust route must rebuild the programs, the file formats and the time loop before it can compare a whole run. After that, no Fortran remains that can share state with the port.

The CPU results of the whole ww3-gpu runtime, with the resident spectrum, and their limits are in its NOAA progress record (Hoffmann 2026e). The DIA port is also published alone as ww3-dia-gpu-v2 (Hoffmann 2026d). That repository compares the port with an earlier CUDA kernel of the same routine.

## 3 Evidence ladders

Each project gives names to its steps. A step in one ledger is not the same evidence as a step in the other ledger.

Table 3: Evidence steps. ww3-gpu counts come from `scripts/port_score.py --write` (profile `bench_small`, 120 × 80 points, 12 h, one thread, gprof, 2026-10-14); chaceon counts come from `just port-summary`.

| Step | ww3-gpu | chaceon |
|---|---|---|
| Code exists | *translated*: 316 routines, 26.2 % of Fortran lines | *translated*: Rust exists, not fully reached |
| A run calls it | *ported*: 72 routines, 99.3 % of profiled run time | *routine*: every captured call is bit-identical on the host and in SPIR-V |
| Whole runs identical, CPU | *regtested*: 14 routines, 48.6 % of run time | *host*: whole runs are `cmp`-identical; 10 080 lines |
| Identical through a GPU API | none | *spirv*: the same files from SPIR-V on CPU Vulkan; 5 189 lines |
| Identical on a real GPU | *gpu*: none | *cuda*: none |

# 4 What each project found

**ww3-gpu: the reference is the optimised code.** Byte identity for the DIA needed one fact that the source text does not give. gfortran expands `x**11` as one chain of multiplications at `-O0` and as a different chain at `-O3`. The two chains disagree on 643 427 of 1 000 000 single-precision inputs. Before this was found, the research log explained the remaining differences as round-off that WW3 amplifies. That explanation was wrong. Thus the reference of the port is the code that the WW3 build produces, read as GIMPLE or disassembly, not the Fortran text. The port has no unit tests of its own. Only the WW3 regression tests, compared by `test.comp`, judge it.

**chaceon: a run with no Fortran in it.** The Rust programs `ww3_grid`, `ww3_strt`, `ww3_prep` and `ww3_shel` run in sequence from the namelists. They write `mod_def.ww3`, the forcing, the gridded and point outputs, the track file and both restarts. All 96 files are byte-identical to the reference. The boundaries give 214 identical comparisons over 34 cases, and the track post-processors give 74. `ww3_shel --threads N` and an MPI build give the same bytes. The files stay identical when `W3SRCE`, `W3XYP3` and `W3KTP3` are compiled to SPIR-V and run on a CPU Vulkan driver. Enumeration cannot reach the $2^{64}$ pairs of `f32` operands. There, a Lean proof states the division fact that the port uses. The proof says nothing about whether Rust equals Fortran, because Fortran has no usable formal semantics.

**Why ww3-gpu has no proof yet, and what comes next.** In this first phase, ww3-gpu adopted no proof assistant (Hoffmann 2026f). There are four reasons:

1. No proof assistant reads Fortran or C++. A proof is about a model that a person writes. It says nothing about the gfortran and nvcc binaries unless somebody shows that the model agrees with them, and that agreement is the open question.
2. Bit equality is a property of two binaries, built with given flags. It is not a property of two source texts.
3. For straight-line IEEE-754 code, equality on every input is decidable. SMT queries in the `QF_FP` theory and sweeps over all $2^{32}$ `float32` inputs answer it in seconds to minutes. An interactive proof takes months for each routine.
4. Agents must write each proof, and a person must review it. This is a second artefact for each routine.

Without a proof, the DIA port has the strongest evidence short of one. It is byte-identical on all 125 entries of the WW3 regression matrix that select it (100 serial, 25 OpenMP). Section 1 of the kernel was also swept over all $2^{32}$ `float32` inputs, with 0 differences (Hoffmann 2026f). Thus the DIA port is validated by the tests of the model that it replaces, on every entry that calls it. This validation is not a formal proof, and the note does not call it one.

A first formal layer is already next to the ports. The Lean 4 package `ocean-waves` (Hoffmann 2026b) states the discretised action balance equation and proves these properties, with no `sorry` and no `axiom`:

- $\sigma^2(k)$ increases strictly with $k$;
- $c/2 \le c_g \le c$;
- the depth CFL condition keeps every Courant number in $[0, 1]$;
- upwind propagation and a DIA-shaped quadruplet exchange conserve action;
- the semi-implicit source step keeps the action positive for every time step.

The package uses exact `Rat` arithmetic on a reduced model: one space dimension, a fixed depth and a Padé model of tanh. Thus it also says nothing about bits. Agda is the next step. Agda has no 32-bit float type, so it cannot prove a fact about bits. Its role is a total, executable reference semantics of each source term. Each table index will have a `Fin` type, so every read is in bounds for every grid, not only for one grid. The sweeps, the solver queries and the WW3 regression tests continue to decide the bits.

# 5 Challenges: the regression tests are large

WW3's regression tests are the only judge of the Kokkos port, and they are large. Table Table 4 gives their size at the fork commit `80644ba9`, counted on 2026-10-18.

Table 4: Size of the WW3 regression matrix. Source: ww3-gpu `regtests/README.md` at `58167d7`, counts from `just ww3-gpu-matrix-list`. The count of all entries is the number of lines that `matrix.base` writes, and a machine section can disable some of them.

| Set of `matrix.base` entries | Entries | Share |
|---|---|---|
| All entries | 758, in 60 tests | 100 % |
| Entries with the DIA (`NL1`) | 310 | 41 % |
| Entries with ST4 physics (`ST4`) | 136 | 18 % |
| Entries that the resident preset can select | 7 (all serial) | 0.9 % |
| Selected regtests of ww3-gpu | 5, and the `ww3_ts1` pair | 0.7 % |

One run of the 310 `NL1` entries takes days. The selected set takes one to three hours on one workstation (Intel i9-14900, 32 hardware threads); this time is an estimate, not a measurement ⚠. Thus a full matrix run cannot follow each commit. ww3-gpu runs the selected set as a guardrail, and it postpones the full matrix to a validation phase. Until then, a static argument supports bit parity: the `-O3` GIMPLE or the disassembly of both compilers.

# 6 What is not claimed

1. Neither project has run the whole model on a real GPU. The PTX of chaceon has never run. ww3-gpu has run one DIA benchmark kernel on an RTX 4090 (Hoffmann 2026d).
2. This note claims no speed-up. The rules of ww3-gpu mark its recorded timings (Hoffmann 2026e) "regtest pending", and chaceon has no timings.
3. On a GPU, bit identity also needs the `expf` and `logf` of the device to return the bits of glibc. ww3-gpu sends its kernels to device copies of the glibc 2.44 functions. These copies equal the host functions on every input that was checked. No device run has compared them yet. Until one does, a fixed tolerance judges a GPU run.
4. The two references are different lines of WW3 (NOAA-EMC and MetOcean). Thus the comparison of the two ports is by routine name, not by source text. Both ledgers contain 85 routines.

# 7 Use of AI

Claude coding agents (Anthropic) produced the C++/Kokkos and Rust code, the shims, the comparison scripts, the investigations and drafts of this text. They worked under the direction of the first author, in the two repositories. Pedro Cruz (GTA, UFRJ) is the advisor and Pedro Veras Guimarães (LabECO, UFSC) is the co-advisor of the first author.

Most of the commits came from Claude Opus. Claude Sonnet also ported a large amount of code without gross errors, in the experience of the first author ⚠. One example is the split of `W3SDS4` into parts that run one sea point per vector lane (ww3-gpu commit `9cef132`, 841 lines added). Its output files were byte-identical to the Fortran of the same executable on the `smoke` and `full` cases.

Each repository publishes the rules, skills and hooks of its agents in its `AGENTS.md` file and its `.claude/` directory. Every number in this note comes from a file of a repository at the commit that the note names. Nothing was run for this note. Each claim was checked on 2026-11-11 against the source and the commit that it names; a ⚠ marks a claim that was not checked against a primary source. The physics of WW3 is used without change; WAVEWATCH III is a registered trademark of NOAA.

**Disclaimer.** Today, both repositories are set up for Claude Code. The authors will continue to work so that any coding agent can use ww3-gpu and chaceon. This includes agents that run local models with Ollama. This work is not complete, and this note does not claim results for other agents or for local models.

# References

Hoffmann, Matheus. 2026a. "chaceon: A Rust Port of WAVEWATCH III." https://github.com/h0ffmann/chaceon.

———. 2026b. “Ocean Waves: The Physics of a Spectral Wave Model in Lean 4.” https://github.com/h0ffmann/proof/tree/b863c15/ocean-waves.
———. 2026c. “WW3 GPU Lab: Agentic Coding and Agentic Research on a Bit-for-Bit GPU Port of WAVEWATCH III.” https://doi.org/10.5281/zenodo.23221351.
———. 2026d. “ww3-dia-gpu-v2: The Whole W3SNL1, Inside WAVEWATCH III, Bit for Bit.” https://github.com/h0ffmann/ww3-dia-gpu-v2/tree/32aa2ef.
———. 2026e. “A Bit-Identical Kokkos Runtime for WAVEWATCH III: CPU Results and Five Speed-Ups.” ww3-gpu, `docs/NOAA_PROGRESS_RECORD_202610.md`, commit 58167d7. https://github.com/h0ffmann/ww3-gpu/blob/58167d7/docs/NOAA_PROGRESS_RECORD_202610.md.
———. 2026f. “ADR-0001: No Proof Language for the GPU Port; SMT Queries and Exhaustive Sweeps Instead.” ww3-gpu, `docs/ADRs/ADR-0001-proof-language.md`, commit 58167d7. https://github.com/h0ffmann/ww3-gpu/blob/58167d7/docs/ADRs/ADR-0001-proof-language.md.
Koldunov, Nikolay V., Suvarchal K. Cheedela, Sergey Danilov, Dmitry Sidorenko, Sebastian Beyer, and Thomas Jung. 2026. “An Ocean Model Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos).” arXiv:2606.11356 [physics.ao-ph]. https://doi.org/10.48550/arXiv.2606.11356.
NOAA Environmental Modeling Center. 2026. “WAVEWATCH IV.” https://github.com/NOAA-EMC/WW4.
Tolman, Hendrik L. 2025. “The WAVEWATCH III Software Modernization Project: Phase I Report.” Office Note 525. College Park, MD: NOAA/NWS/NCEP. https://doi.org/10.25923/h7j3-1h25.